

Prueba Tecnica: Associate Data Engineer: Brenda Nicole Henríquez Amaya

Problema de negocio

El objetivo de este proyecto es diseñar e implementar un proceso de ingeniería de datos (ETL) que resuelva una necesidad operativa y analítica dividida en tres fases consecutivas:

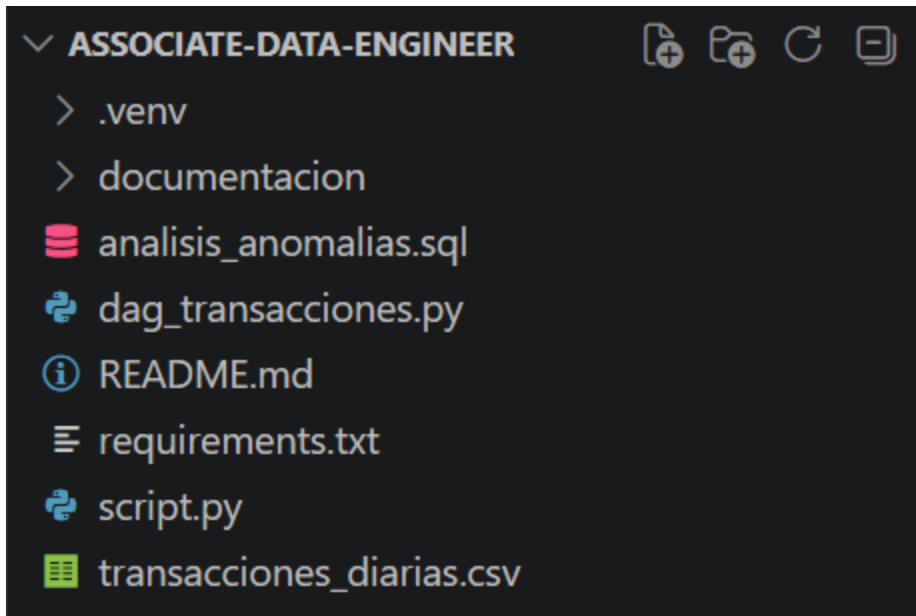
Procesamiento y Carga (Python): Leer un conjunto de datos crudos denominado transacciones_diarias.csv, limpiarlo bajo reglas de calidad específicas (manejo de duplicados e imputación de valores nulos) y estructurarlo bajo un modelo relacional de Esquema Snowflake (Esquema de Copo de Nieve). Finalmente, los datos procesados deben cargarse de forma automatizada en una base de datos en la nube (Supabase).

Análisis de Anomalías (SQL): Desarrollar una consulta analítica que explore el modelo relacional cargado. El propósito es identificar patrones inusuales o posibles fraudes financieros, específicamente detectando aquellos usuarios cuyos montos de gasto aprobados se hayan incrementado de forma abrupta en comparación con su transacción inmediata anterior.

Propuesta de Orquestación (Apache Airflow): Diseñar un script conceptual de automatización que programe la ejecución de todo el flujo de trabajo para que corra diariamente a las 11:30 PM, garantizando que el análisis analítico solo se ejecute si la carga de datos previa concluyó con éxito.

Estructura del Proyecto

Para mantener un entorno de desarrollo limpio, el proyecto lo he organizado bajo la siguiente estructura de archivos y carpetas visibles en el espacio de trabajo:



Descripción

- **Entorno Virtual (`.venv/`):** Carpeta técnica que contiene la instalación aislada de Python y las librerías del proyecto. Su uso garantiza que las dependencias del proyecto no se mezclen con el sistema operativo global, asegurando la estabilidad del pipeline.
- **Carpeta de Documentación (`documentacion/` y `README.md`):** Espacios dedicados exclusivamente al almacenamiento de manuales, diagramas de arquitectura y la descripción general del repositorio para facilitar la transferencia de conocimiento.
- **Scripts de python y analisis de sql (`script.py` y `analisis_anomalias.sql`):** Archivos donde se centraliza el procesamiento de datos. Se mantiene una separación estricta de tecnologías: Python se encarga de la limpieza y carga estructural, mientras que SQL ejecuta la explotación analítica directamente en el motor de la base de datos.
- **Orquestación (`dag_transacciones.py`):** Archivo independiente que contiene la lógica conceptual para automatizar el orden y los tiempos de ejecución del flujo de datos.
- **Datos crudos (`transacciones_diarias.csv`):** Ubicado en la raíz del proyecto para permitir una lectura directa mediante rutas relativas dentro del código, evitando fallos de localización al mover el proyecto de computadora.

Entregable 1: Script de Limpieza y Carga (`script.py`)

Este archivo contiene el desarrollo en Python encargado de la Fase de Extracción, Transformación y Carga (ETL). Su objetivo principal es recibir el archivo plano `transacciones_diarias.csv`, aplicar transformaciones basadas en reglas de calidad de datos, estructurar la información bajo un modelo relacional de tipo Esquema Snowflake y cargar el resultado de forma directa y segura en la instancia de base de datos en la nube (Supabase).

1.1 Módulos de Dependencias y Conexión

El inicio del script importa las librerías necesarias y define la ruta de acceso al motor de base de datos.

```
import pandas as pd
from sqlalchemy import create_engine, text

# Cadena de conexión estricta hacia La instancia de PostgreSQL en Supabase
con cifrado SSL
DATABASE_URL =
"postgresql://postgres:basededatos123@db.pz1knfqntvljoduxegk.supabase.co:543
sslmode=require"
```

1.2 Función de Transformación y Normalización (procesar_y_normalizar_snowflake)

Esta función procesa el archivo crudo y aplica consecutivamente las tres primeras reglas de negocio exigidas:

```
def procesar_y_normalizar_snowflake(filepath):
    # Lectura del archivo origen en formato CSV utilizando la ruta
    relativa
    df = pd.read_csv(filepath)

    # REGLA DE NEGOCIO 1: Limpieza de registros duplicados basados en el
    identificador único de transacción.
    df = df.drop_duplicates(subset=['id_transaccion'], keep='first')

    # REGLA DE NEGOCIO 2: Gestión de valores nulos.
    # Si el monto es ausente y el estado de la transacción fue
    'rechazada', se imputa el valor a 0.0.
    condicion_nulo_rechazado = (df['monto_usd'].isna()) &
(df['estado_transaccion'] == 'rechazada')
    df.loc[condicion_nulo_rechazado, 'monto_usd'] = 0.0

    # REGLA DE NEGOCIO 3: Clasificación de transacciones sospechosas.
    # Se marcan como verdaderas (True) aquellas de carácter internacional
    con montos que superen los $1500 USD.
    df['es_monto_inusual'] = (df['monto_usd'] > 1500) &
(df['tipo_comercio'] == 'internacional')
```

Posteriormente, dentro de la misma función, se ejecuta la **descomposición relacional** para dar forma al Esquema Snowflake, el cual separa las entidades de negocio para eliminar la redundancia:

```
# ESTRUCTURACIÓN DEL ESQUEMA SNOWFLAKE:

# 1. Dimensión Clientes: Extrae los identificadores únicos de cada
cliente presente en los datos.
dim_clientes =
df[['id_cliente']].drop_duplicates().reset_index(drop=True)
```

2. Dimensión Comercios: Aísla el tipo de comercio y genera un código numérico secuencial (Llave Primaria).

```
dim_comercios =
df[['tipo_comercio']].drop_duplicates().reset_index(drop=True)
dim_comercios['id_comercio'] = dim_comercios.index + 1
```

3. Vinculación: Cruzamos el DataFrame original con la dimensión de comercios para asociar a cada registro su respectivo ID.

```
df_fact = df.merge(dim_comercios, on='tipo_comercio', how='left')
```

4. Tabla de Hechos (Fact Table): Se seleccionan únicamente Las Llaves foráneas y Las métricas numéricas.

```
fact_transacciones = df_fact[[
    'id_transaccion', 'id_cliente', 'id_comercio', 'fecha_hora',
    'monto_usd', 'estado_transaccion', 'es_monto_inusual'
]]
```

```
return dim_clientes, dim_comercios, fact_transacciones
```

1.3 Función de Carga (cargar_a_supabase_snowflake)

Esta función interactúa de forma directa con Supabase. Primero remueve de manera limpia los datos antiguos y luego inserta las nuevas tablas estructuradas, asegurando el cumplimiento de las restricciones relacionales de la base de datos:

```
def cargar_a_supabase_snowflake(dim_cli, dim_com, fact_trans):
    engine = create_engine(DATABASE_URL)

    # Eliminación segura de tablas previas para evitar colisiones de datos
    # o errores de existencia.
    try:
        with engine.connect() as connection:
            connection.execute(text("DROP TABLE IF EXISTS
fact_transacciones CASCADE;"))
            connection.execute(text("DROP TABLE IF EXISTS dim_clientes
CASCADE;"))
            connection.execute(text("DROP TABLE IF EXISTS dim_comercios
CASCADE;"))
            connection.commit()
    except Exception as e:
        pass

    try:
        # Inserción de Los DataFrames de Pandas directamente en tablas de
        # La base de datos PostgreSQL
        dim_cli.to_sql('dim_clientes', engine, if_exists='replace',
index=False)
        dim_com.to_sql('dim_comercios', engine, if_exists='replace',
index=False)
        fact_trans.to_sql('fact_transacciones', engine,
if_exists='replace', index=False)
```

```

        # Ejecución de comandos DDL para definir formalmente el esquema de
        base de datos
        with engine.connect() as connection:
            # Definición de Llaves Primarias (Garantizan unicidad de
            filas)

            connection.execute(text("ALTER TABLE dim_clientes ADD PRIMARY
            KEY (id_cliente);"))
            connection.execute(text("ALTER TABLE dim_comercios ADD PRIMARY
            KEY (id_comercio);"))
            connection.execute(text("ALTER TABLE fact_transacciones ADD
            PRIMARY KEY (id_transaccion);"))

            # Definición de Llaves Foráneas (Establecen la integridad
            referencial del Esquema Snowflake)
            connection.execute(text("""
                ALTER TABLE fact_transacciones
                ADD CONSTRAINT fk_cliente FOREIGN KEY (id_cliente)
            REFERENCES dim_clientes(id_cliente),
                ADD CONSTRAINT fk_comercio FOREIGN KEY (id_comercio)
            REFERENCES dim_comercios(id_comercio);
            """))
            connection.commit()
            print("¡ÉXITO EN EL PIPELINE! El modelo Snowflake fue
            estructurado y poblado correctamente.")
        except Exception as e:
            print(f"❌ Error crítico durante la carga en la base de datos:
            {e}")

```

1.4 Bloque de Ejecución Principal (`__main__`)

Determina el punto de entrada del script, llamando de forma ordenada al proceso de transformación de datos y posteriormente a la carga final.

```

if __name__ == "__main__":
    # Definición del archivo fuente
    archivo_origen = "transacciones_diaras.csv"

    # Orquestación local de las funciones
    dim_cli, dim_com, fact_trans =
    procesar_y_normalizar_snowflake(archivo_origen)
    cargar_a_supabase_snowflake(dim_cli, dim_com, fact_trans)

```

Entregable 2: Consulta Analítica de Anomalías (`analisis_anomalias.sql`)

Este entregable contiene el desarrollo en lenguaje SQL encargado de la **Fase 2 (Análisis de Anomalías)**. Su propósito es interrogar el modelo relacional estructurado en Supabase para identificar de manera automatizada patrones de comportamiento inusuales o potenciales fraudes financieros, cumpliendo estrictamente con la **Regla de Negocio 4**: detectar transacciones aprobadas cuyo monto actual sea igual o mayor a cinco veces el monto de la transacción inmediatamente anterior del mismo cliente.

Codigo del archivo:

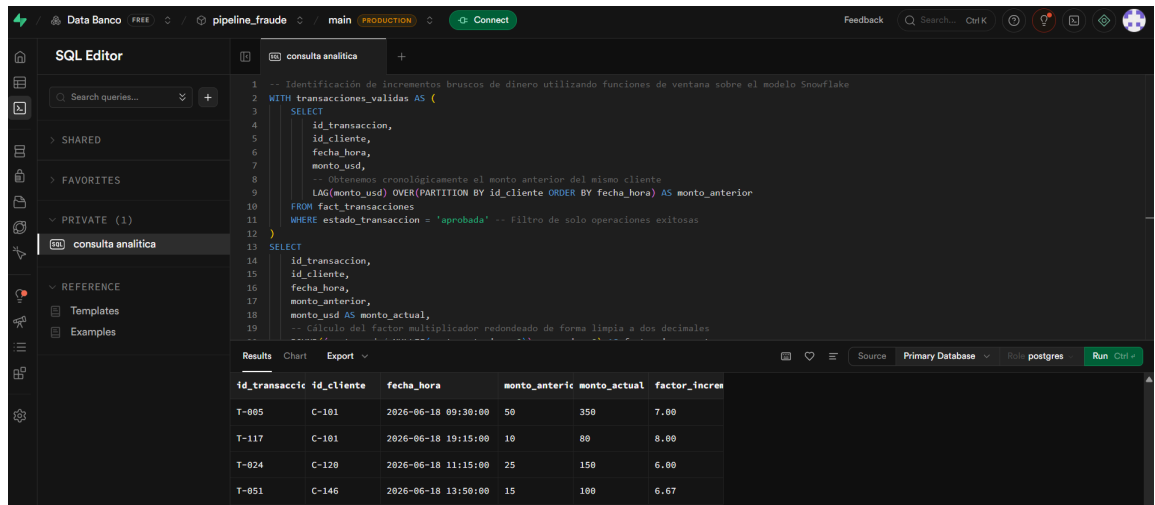
```
-- Identificación de incrementos bruscos de dinero utilizando funciones de
-- ventana sobre el modelo Snowflake
WITH transacciones_validas AS (
    SELECT
        id_transaccion,
        id_cliente,
        fecha_hora,
        monto_usd,
        -- Obtenemos cronológicamente el monto anterior del mismo cliente
        LAG(monto_usd) OVER(PARTITION BY id_cliente ORDER BY fecha_hora)
    AS monto_anterior
    FROM fact_transacciones
    WHERE estado_transaccion = 'aprobada' -- Filtro de solo operaciones
    exitosas
)
SELECT
    id_transaccion,
    id_cliente,
    fecha_hora,
    monto_anterior,
    monto_usd AS monto_actual,
    -- Cálculo del factor multiplicador redondeado de forma limpia a dos
    decimales
    ROUND((monto_usd / NULLIF(monto_anterior, 0))::numeric, 2) AS
    factor_incremento
FROM transacciones_validas
WHERE monto_anterior IS NOT NULL
    AND monto_usd >= (5 * monto_anterior) -- Criterio: incremento igual o
    mayor a 5 veces
ORDER BY id_cliente, fecha_hora;
```

¿Qué hace el código?

1. **Uso de Expresiones de Tabla Comunes (WITH ... AS)**: Se define una subconsulta temporal nombrada `transacciones_validas`. Esto segmenta la lógica en dos pasos (aislamiento de datos históricos y filtrado final), mejorando significativamente la legibilidad y el rendimiento de la consulta sobre el motor PostgreSQL.
2. **La Función de Ventana Analítica (LAG)**: La instrucción clave es `LAG(monto_usd) OVER(PARTITION BY id_cliente ORDER BY fecha_hora)`.

- **LAG(monto_usd)** : Permite acceder a una fila anterior dentro del mismo conjunto de datos sin necesidad de realizar costosos cruces de tablas (*Self-Joins*).
 - **PARTITION BY id_cliente** : Agrupa el historial de transacciones de forma aislada para cada cliente, garantizando que el cálculo de un usuario no se mezcle con el de otro.
 - **ORDER BY fecha_hora** : Organiza de manera estrictamente cronológica las operaciones, asegurando que el valor recuperado por **LAG** corresponda exactamente a la transacción inmediata anterior en el tiempo.
3. **Control de Errores Operativos (NULLIF)**: En la fase de selección final se utiliza **NULLIF(monto_anterior, 0)** . Esta es una medida de seguridad en ingeniería de datos que transforma cualquier valor de "cero" en un valor nulo (**NULL**), evitando de forma matemática el error crítico de división por cero (*Division by Zero Error*) que detendría la ejecución del pipeline.
4. **Conversión y Redondeo Estricto (ROUND)**: La operación **ROUND(...)::numeric, 2)** realiza una conversión explícita de datos (*casting*) hacia el tipo **numeric** requerida por PostgreSQL, permitiendo truncar y redondear el factor multiplicador a exactamente dos unidades decimales, garantizando un reporte analítico limpio (por ejemplo, transformando **6.66666...** en **6.66**).
5. **Evaluación del Umbral Financiero (WHERE)**: La cláusula de salida filtra el universo de transacciones evaluando la condición de negocio: **monto_usd >= (5 * monto_anterior)** . Solo aquellos registros que hayan cumplido con un incremento de cinco o más veces respecto a su punto de partida histórico se consolidan como anomalías sospechosas.

Captura del codigo desde Supabase y el resultado



The screenshot shows the Supabase SQL Editor interface. The query being executed is as follows:

```

1 -- Identificación de incrementos bruscos de dinero utilizando funciones de ventana sobre el modelo Snowflake
2 WITH transacciones_validas AS (
3   SELECT
4     id_transaccion,
5     id_cliente,
6     fecha_hora,
7     monto_usd,
8     -- Obtenemos cronológicamente el monto anterior del mismo cliente
9     LAG(monto_usd) OVER(PARTITION BY id_cliente ORDER BY fecha_hora) AS monto_anterior
10  FROM fact_transacciones
11  WHERE estado_transaccion = 'aprobada' -- Filtro de solo operaciones exitosas
12 )
13 SELECT
14   id_transaccion,
15   id_cliente,
16   fecha_hora,
17   monto_anterior,
18   monto_usd AS monto_actual,
19   -- Cálculo del factor multiplicador redondeado de forma limpia a dos decimales
20   ROUND(monto_usd / monto_anterior, 2) AS factor_incremento
21 FROM transacciones_validas
22 WHERE monto_usd >= (5 * monto_anterior)

```

The results of the query are displayed in a table with the following columns: **id_transacción**, **id_cliente**, **fecha_hora**, **monto_anterior**, **monto_actual**, and **factor_incremento**.

id_transacción	id_cliente	fecha_hora	monto_anterior	monto_actual	factor_incremento
T-005	C-101	2026-06-18 09:30:00	50	350	7.00
T-117	C-101	2026-06-18 19:15:00	10	80	8.00
T-024	C-120	2026-06-18 11:15:00	25	150	6.00
T-051	C-146	2026-06-18 13:50:00	15	100	6.67

Entregable 3. (README.md)

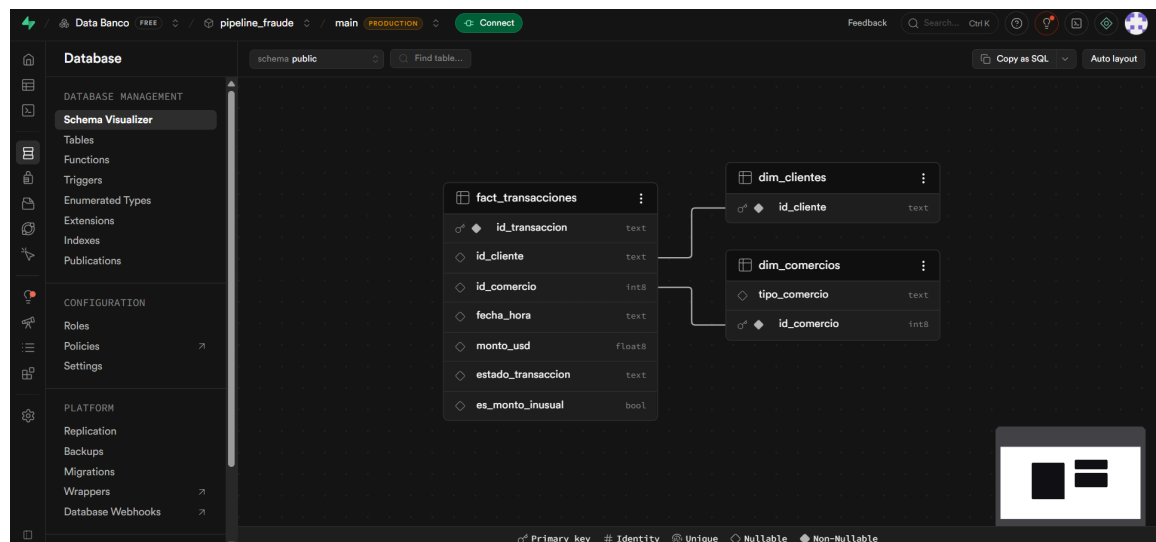
1. Justificación de Calidad de Datos

Para que el equipo de analítica pueda trabajar correctamente y descubrir patrones o fraudes sin errores, los datos deben estar limpios. Aplicar las reglas de limpieza es obligatorio por tres razones:

- **Evitar cálculos inflados (Manejo de Duplicados):** Si dejamos transacciones repetidas con el mismo ID, los reportes finales mostrarán que se gastó más dinero del real o que hubo más ventas de las que verdaderamente sucedieron, arruinando cualquier análisis.
- **Evitar que los programas se traben (Manejo de Nulos):** Las herramientas de análisis matemático no entienden qué hacer cuando ven un espacio vacío o un valor nulo (NaN) en la columna de dinero. Al colocarle un `0.0` a las transacciones que fueron rechazadas y venían vacías, nos aseguramos de que las operaciones matemáticas de los demás compañeros funcionen sin dar errores.
- **Facilitar el trabajo futuro (Clasificación de Montos Inusuales):** Dejar marcado desde el principio si una transacción es internacional y mayor a \$1500 ayuda a que los analistas no tengan que hacer filtros pesados cada vez que busquen movimientos sospechosos; el dato ya va listo y etiquetado para ellos.

2. Arquitectura y Flujo de Datos

El camino que siguen los datos está diseñado para ser seguro y ordenado.



Librerías utilizadas y el por qué se eligieron:

- **Pandas (Estructura en memoria):** Es la herramienta principal en Python para leer archivos CSV. Se eligió porque permite hacer la limpieza, borrar los duplicados y rellenar los datos vacíos de forma muy rápida y con pocas líneas de código.
- **SQLAlchemy y Psycopg2 (Conexión y Envío):** Son los encargados de crear el puente de comunicación hacia Supabase. Se eligieron porque se aseguran de que la información viaje protegida por internet y traducen los datos de Python al lenguaje que entiende la base de datos, creando las uniones y llaves necesarias de manera automática.

Acceso al Código Fuente - Repositorio GitHub

Puede acceder y revisar el proyecto completo a través del siguiente enlace oficial:

Repositorio del Proyecto: [AmayaNicole / Associate-Data-Engineer](#)